Assessment 2A: Case Study Design

UO System Design and Realisation
INFO 2026

Manny Asbanu
a1926920

Class Level Diagram

See attached PDF: 2A-class-diagram.pdf

Key Assumptions and Design Impact

Key assumption	How the assumption shaped the design
Paper titles are unique within the loaded programme.	Paper titles are used as identifiers in the Session Access index and stored in Schedule, rather than copying complete Paper objects. This makes comparison and merging simple and avoids duplicate data. CsvProgrammeImporter rejects duplicate titles with ValueError. If multiple conference files later introduce title collisions, identifier changes can be confined mainly to the domain and Session Access components.
A session name uniquely identifies one session block within the loaded programme.	SessionAccess stores sessions in dict[str, Session], keyed by session name, because ScheduleService.list_papers() accepts only session_name. If sessions with the same name must later exist across different conferences or days, Session Access could adopt a composite identifier without changing the comparison or summary components.

All rows sharing a session name have the same day, track, room, start time, and duration.	Shared information is stored once in Session, while Paper contains only its title and authors. This avoids duplicated session data and establishes a composition relationship from Session to Paper. Conflicting metadata is treated as malformed input rather than creating an ambiguous session.
The supplied sessions.csv is normally valid, but invalid input must still be handled at the system boundary.	CsvProgrammeImporter validates required columns, dates, times, positive durations, malformed rows, inconsistent sessions, and duplicate titles. It raises ValueError as required by the façade contract. Validation remains within the importer, preventing CSV concerns from leaking into SessionAccess or ScheduleService.
Importing a programme replaces the current catalogue and occurs before users create schedules.	SessionAccess.load_sessions() parses the entire file before atomically replacing its internal session and paper indexes. A failed import therefore leaves the previous catalogue unchanged. Support for combining multiple files can later be introduced through another IProgrammeImporter implementation without changing catalogue consumers.
A header only CSV represents a valid empty programme, while a completely empty file is missing its required columns.	A header only file loads zero papers and produces an empty session list. A zero byte file raises ValueError. This provides defined behaviour for the rubric's empty file boundary case while remaining consistent with the required column contract.
Saved schedules persist for the lifetime of the running application, rather than across application restarts.	The provided application creates one ScheduleService and reuses it, so ScheduleAccess stores schedules in memory. All consumers depend on IScheduleAccess, allowing a database or file backed implementation to be introduced later without changing the comparator, summary generator, or façade methods.
Schedule names are globally unique within the current application, and no separate user account model is required.	ScheduleAccess indexes schedules by name. add_to_schedule() creates a schedule when the name is new, while save_new() rejects an existing name to avoid silently overwriting another schedule. If user accounts are added later, ownership can be incorporated into the Schedule Access implementation and storage key.

Adding the same paper to a schedule more than once has no effect.	Schedule enforces uniqueness while preserving insertion order. This prevents repeated papers in schedule displays, comparisons, summaries, and joint schedules. Comparison results are also deterministic: merged schedules contain schedule A's papers followed by papers unique to schedule B.
Unknown names follow the fixed façade contract where specified.	list_papers() and list_schedule_papers() return empty lists when the requested item does not exist. add_to_schedule() raises ValueError for an unknown paper. Because missing schedule behaviour is not specified for comparison or summary, those operations raise ValueErrorrather than silently treating a misspelled name as an empty schedule.
The fixed summarise() contract is authoritative where it differs from the general use case wording.	The summary contains "paper_count", "tracks", and "total_minutes" exactly as defined in schedule_service.py, rather than adding an unsupported "sessions" key. Track counts are calculated per selected paper. total_minutes sums the associated session duration for each selected paper, following the provided docstring. This calculation is isolated in SummaryGenerator, so it can be changed to count each distinct session once if that interpretation is later confirmed.
Dates and times have temporal meaning, while room identifiers are opaque strings.	Session stores the day and start time using date/time types and duration as an integer, supporting future clash detection without changing CSV parsing throughout the system. The room is retained as a string because no physical room mapping is required. Filtering, clash detection, and schedule export can operate through the existing Session and Schedule Access abstractions.

Design Principles

Single Responsibility Principle

Each class has one main role. *CsvProgrammeImporter* reads and validates CSV data. *SessionAccess* manages programme data, while *ScheduleAccess* manages saved schedules. *ScheduleComparator* compares schedules, and *SummaryGenerator* creates summaries. *ScheduleService* only coordinates these components and formats results for the web application. This keeps the classes focused and allows each part to be tested separately.

Open–Closed Principle

The main components are defined through interfaces. New implementations can be added without changing the classes that use them. For example, an in-memory schedule store could later be replaced with database storage through *IScheduleAccess*. Another file format could be supported through *IProgrammeImporter*.
Filtering, clash detection, and schedule exporting could also be added as separate components using the existing access interfaces. Interfaces are only used at important component boundaries to avoid unnecessary complexity.

Dependency Inversion and Injection

Components depend on interfaces rather than concrete classes. *ScheduleComparator* depends on *IScheduleAccess*, while *SummaryGenerator* depends on both *IScheduleAccess* and *ISessionAccess*.
These dependencies are passed through constructors. This makes it possible to supply fake implementations during testing. *ScheduleService.__init__()* creates and connects the real implementations because the provided application requires a no-argument constructor.

Design Patterns

ScheduleService uses the Facade pattern. It gives the web application one entry point and hides the classes behind it.
IProgrammeImporter uses the Strategy pattern. *SessionAccess* can use different importers without changing its storage logic.
The access interfaces also act as Repository-style abstractions. Other components retrieve data through these interfaces instead of directly accessing files or dictionaries.

Design Scope

The design uses more than one class because the task requires separate components. However, extra layers such as factories, commands, or separate storage repositories were not added. This keeps the design flexible without making it unnecessarily complex.